

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине Администрирование платформ на ОС Linux

Тема работы Docker

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ.....	4
1.1 Общая структура проекта.....	4
1.2 Сервис Nginx	8
1.3 Сервис LibreSpeed.....	11
1.4 Сервис PostgreSQL	12
1.5 Сервис резервного копирования базы данных	13
1.6 Сервис ротации и отправки логов nginx.....	16
2 ТЕСТИРОВАНИЕ	21
2.1 Проверка работоспособности сайта	21
2.2 Отказоустойчивость при падении сервисов	22
2.3 Мониторинг состояния контейнеров	23
2.4 Тестирование резервного копирования	25
2.4.1 Резервное копирование базы данных	25
2.4.2 Ротация и отправка логов nginx	26
3 ВОПРОСЫ	27
ЗАКЛЮЧЕНИЕ	28

ВВЕДЕНИЕ

Цель работы: научиться переводить инфраструктуру, развёрнутую на процессах Linux, в декларативное окружение на базе Docker Compose с использованием собственных Dockerfile.

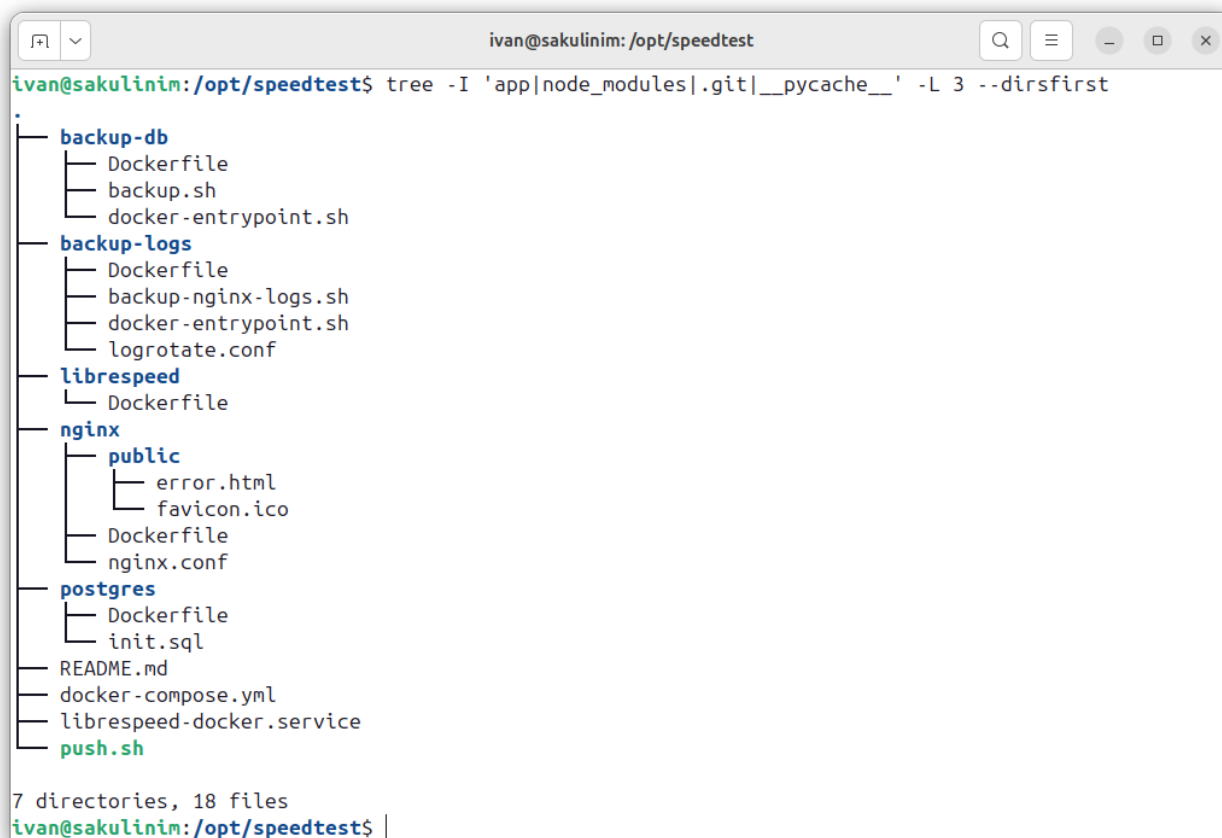
Задачи:

- разработать Dockerfile для каждого компонента: nginx, PostgreSQL, LibreSpeed (Node.js), сервисы резервного копирования БД и логов;
- описать единый docker-compose.yml;
- вынести изменяемые параметры в файл .env;
- создать systemd-юнит для автозапуска при загрузке сервера;
- проверить работоспособность и отказоустойчивость системы.

1 ХОД РАБОТЫ

1.1 Общая структура проекта

Исходные файлы проекта организованы в виде директорий для каждого сервиса, содержащих Dockerfile и необходимые конфигурации. На рисунке 1 представлена структура проекта (исключая содержимое папки **app** с исходным кодом LibreSpeed).



```
ivan@sakulinim: /opt/speedtest
ivan@sakulinim: /opt/speedtest$ tree -I 'app|node_modules|.git|__pycache__' -L 3 --dirsfirst
.
├── backup-db
│   ├── Dockerfile
│   ├── backup.sh
│   └── docker-entrypoint.sh
├── backup-logs
│   ├── Dockerfile
│   ├── backup-nginx-logs.sh
│   ├── docker-entrypoint.sh
│   └── logrotate.conf
├── librespeed
│   └── Dockerfile
├── nginx
│   ├── public
│   │   ├── error.html
│   │   └── favicon.ico
│   ├── Dockerfile
│   └── nginx.conf
├── postgres
│   ├── Dockerfile
│   └── init.sql
├── README.md
├── docker-compose.yml
├── librespeed-docker.service
└── push.sh

7 directories, 18 files
ivan@sakulinim: /opt/speedtest$
```

Рисунок 1 — Дерево файлов проекта

Ключевым файлом, описывающим все сервисы, является `docker-compose.yml` (рисунок 2).

```
1 services:
2   postgres:
3     build: ./postgres
4     container_name: postgres
5     restart: always
```

```

6     environment:
7         POSTGRES_HOST: ${POSTGRES_HOST}
8         POSTGRES_PORT: ${POSTGRES_PORT}
9         POSTGRES_USER: ${POSTGRES_USER}
10        POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
11        POSTGRES_DB: ${POSTGRES_DATABASE}
12    volumes:
13        - pg_data:/var/lib/postgresql/data
14    healthcheck:
15        test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
16            ↪ ${POSTGRES_DATABASE}"]
17        interval: 10s
18        timeout: 5s
19        retries: 5
20    libspeed-1: &libspeed
21        container_name: libspeed-1
22        build: ./libspeed
23        restart: always
24        environment:
25            PG_HOST: ${POSTGRES_HOST}
26            PG_PORT: ${POSTGRES_PORT}
27            PG_USER: ${POSTGRES_USER}
28            PG_PASSWORD: ${POSTGRES_PASSWORD}
29            PG_DATABASE: ${POSTGRES_DATABASE}
30        depends_on:
31            postgres:
32                condition: service_healthy
33
34    libspeed-2:
35        container_name: libspeed-2
36        <<: *libspeed
37
38    libspeed-3:
39        container_name: libspeed-3
40        <<: *libspeed
41
42    nginx:
43        build: ./nginx
44        container_name: nginx

```

```

45     restart: always
46     ports:
47         - "80:80"
48         - "443:443"
49     deploy:
50         resources:
51             limits:
52                 memory: 512M
53     volumes:
54         - nginx_logs:/var/log/nginx
55         - /etc/letsencrypt:/etc/letsencrypt:ro
56     depends_on:
57         - librespeed-1
58         - librespeed-2
59         - librespeed-3
60
61     backup-db:
62         build: ./backup-db
63         container_name: backup-db
64         environment:
65             POSTGRES_HOST: ${POSTGRES_HOST}
66             POSTGRES_PORT: ${POSTGRES_PORT}
67             POSTGRES_USER: ${POSTGRES_USER}
68             POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
69             POSTGRES_DATABASE: ${POSTGRES_DATABASE}
70             AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID}
71             AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY}
72             AWS_DEFAULT_REGION: ${AWS_DEFAULT_REGION}
73             AWS_ENDPOINT_URL: ${AWS_ENDPOINT_URL}
74             S3_BUCKET_DB: ${S3_BUCKET_DB}
75             BACKUP_DB_CRON: ${BACKUP_DB_CRON}
76         depends_on:
77             postgres:
78                 condition: service_healthy
79         restart: unless-stopped
80
81     backup-logs:
82         build: ./backup-logs
83         container_name: backup-logs
84         environment:

```

```

85     AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID}
86     AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY}
87     AWS_DEFAULT_REGION: ${AWS_DEFAULT_REGION}
88     AWS_ENDPOINT_URL: ${AWS_ENDPOINT_URL}
89     S3_BUCKET_LOGS: ${S3_BUCKET_LOGS}
90     BACKUP_LOGS_CRON: ${BACKUP_LOGS_CRON}
91     BACKUP_LOGS_VOLUME: /var/log/nginx
92     restart: unless-stopped
93     volumes:
94         - nginx_logs:/var/log/nginx
95         - /var/run/docker.sock:/var/run/docker.sock:ro
96
97 volumes:
98     pg_data:
99     nginx_logs:

```

Рисунок 2 — Файл docker-compose.yml

Переменные окружения, используемые в `docker-compose.yml`, вынесены в файл `.env` (рисунок 3).

```

1  # PostgreSQL
2  POSTGRES_HOST=postgres
3  POSTGRES_PORT=5432
4  POSTGRES_USER=app_user
5  POSTGRES_PASSWORD=***
6  POSTGRES_DATABASE=librespeed
7  # AWS S3
8  AWS_ACCESS_KEY_ID=***
9  AWS_SECRET_ACCESS_KEY=***
10 AWS_DEFAULT_REGION=ru-3
11 AWS_ENDPOINT_URL = https://s3.ru-3.storage.selcloud.ru
12 # Backups
13 S3_BUCKET_DB=s3://sakulinim.speedtest.logs/db/
14 BACKUP_DB_CRON= 0 22 * * *
15 S3_BUCKET_LOGS=s3://sakulinim.speedtest.logs/
16 BACKUP_LOGS_CRON=0 22 * * *

```

Рисунок 3 — Файл .env с переменными окружения

Для автоматического запуска стека при загрузке операционной системы создан systemd-юнит `librespeed-docker.service` (рисунок 4).

```
1 [Unit]
2 Description=LibreSpeed via Docker Compose by Ivan Sakulin
3 Requires=docker.service
4 After=docker.service
5 [Service]
6 Type=oneshot
7 RemainAfterExit=yes
8 WorkingDirectory=/opt/speedtest
9 ExecStart=/usr/bin/docker compose up -d
10 ExecStop=/usr/bin/docker compose down
11 [Install]
12 WantedBy=multi-user.target
```

Рисунок 4 — Systemd-юнит для автозапуска Docker Compose

1.2 Сервис Nginx

Nginx выполняет роль обратного прокси-сервера, балансируя запросы между тремя экземплярами приложения LibreSpeed. Конфигурация включает поддержку SSL, кастомную страницу ошибок, блокировку нежелательных ботов и набор заголовков безопасности.

Сборка образа описана в `nginx/Dockerfile` (рисунок 5).

```
1 FROM nginx:alpine
2 RUN rm /etc/nginx/conf.d/default.conf \
3     && rm -f /var/log/nginx/access.log /var/log/nginx/error.log \
4     && touch /var/log/nginx/access.log /var/log/nginx/error.log \
5     && chown nginx:nginx /var/log/nginx/*.log
6 COPY nginx.conf /etc/nginx/conf.d/
7 COPY ./public /public
```

Рисунок 5 — Dockerfile для Nginx

Конфигурационный файл `nginx.conf` представлен на рисунке 6.

```

1  server_tokens off;
2  access_log /var/log/nginx/access.log;
3  error_log /var/log/nginx/error.log;
4  map $http_user_agent $bad_bot {
5      default 0;
6      ~*msnbot|scrapbot|wget|python|java|httpclient|okhttp|winhttp|HTTrack|
7      WebZIP|FlashGet|GetRight|qqmail|curl 1;
8  }
9  upstream backend_servers {
10     server libspeed-1:3000;
11     server libspeed-2:3000;
12     server libspeed-3:3000;
13 }
14 server {
15     listen 80;
16     listen 443 ssl;
17     server_name otz.duckdns.org;
18     if ($host != otz.duckdns.org) {
19         return 444;
20     }
21     if ($scheme = 'http') {
22         return 301 https://$host$request_uri;
23     }
24     add_header Strict-Transport-Security 'max-age=31536000; includeSubDomains;
25     ↪ preload';
26     add_header X-Frame-Options SAMEORIGIN always;
27     add_header X-Content-Type-Options nosniff;
28     add_header X-XSS-Protection "1; mode=block";
29     add_header Referrer-Policy "strict-origin-when-cross-origin" always;
30     add_header Permissions-Policy "geolocation=(), camera=(), microphone=()"
31     ↪ always;
32     add_header Content-Security-Policy "default-src 'self'; script-src 'self'
33     ↪ 'unsafe-inline'; style-src 'self' 'unsafe-inline';" always;
34     if ($bad_bot) {
35         return 403;
36     }
37     proxy_intercept_errors on;
38     error_page 403 404 500 502 503 504 /error.html?code=$status;
39     client_header_timeout 10s;

```

```

37     client_body_timeout 10s;
38     send_timeout 10s;
39     keepalive_timeout 10s;
40     reset_timedout_connection on;
41     client_body_buffer_size 128k;
42     client_header_buffer_size 1k;
43     client_max_body_size 128m;
44     large_client_header_buffers 2 1k;
45     location / {
46         limit_except GET POST {
47             deny all;
48         }
49         proxy_pass http://backend_servers;
50         proxy_set_header Host $host;
51         proxy_set_header X-Real-IP $remote_addr;
52         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
53         proxy_set_header X-Forwarded-Proto $scheme;
54     }
55     location = /error.html {
56         root /public;
57         internal;
58         ssi on;
59     }
60     location = /favicon.ico {
61         root /public;
62         expires 30d;
63         add_header Cache-Control "public, immutable";
64     }
65     ssl_certificate /etc/letsencrypt/live/otz.duckdns.org/fullchain.pem; #
66     ↪ managed by Certbot
67     ssl_certificate_key /etc/letsencrypt/live/otz.duckdns.org/privkey.pem; #
68     ↪ managed by Certbot
69     include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
70     ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot
71 }

```

Рисунок 6 — Конфигурация Nginx

Статические ресурсы (favicon.ico, error.html) помещены в поддиректорию public/ и копируются в образ. Содержимое error.html (с поддержкой SSI) показано на рисунке 7.

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Error <!--# echo var="arg_code" default="unknown" --></title>
5      <style>
6          body { font-family: Arial, sans-serif; text-align: center; padding:
              ↪ 50px; }
7          h1 { font-size: 48px; margin-bottom: 20px; }
8      </style>
9  </head>
10 <body>
11     <h1>Error <!--# echo var="arg_code" default="500" --></h1>
12     <p>An error occurred while processing your request.</p>
13     <p><a href="/">Go back to the main page</a></p>
14 </body>
15 </html>
```

Рисунок 7 — Кастомная страница ошибок

1.3 Сервис LibreSpeed

Dockerfile (рисунок 8) копирует исходный код приложения LibreSpeed на Node.js и устанавливает зависимости.

```
1 FROM node:18-alpine
2 WORKDIR /usr/src/app
3 COPY ./app/ ./
4 RUN npm install --production
5 CMD ["node", "src/SpeedTest.js", "0.0.0.0", "3000"]
```

Рисунок 8 — Dockerfile для LibreSpeed

1.4 Сервис PostgreSQL

База данных PostgreSQL используется для хранения телеметрии LibreSpeed. Dockerfile сервиса приведён на рисунке 9. Инициализация таблиц выполняется скриптом `init.sql` (рисунок 10), который копируется в точку входа образа.

```
1 FROM postgres:15-alpine
2 COPY init.sql /docker-entrypoint-initdb.d/
```

Рисунок 9 — Dockerfile для PostgreSQL

```
1 SET statement_timeout = 0;
2 SET lock_timeout = 0;
3 SET idle_in_transaction_session_timeout = 0;
4 SET client_encoding = 'UTF8';
5 SET standard_conforming_strings = on;
6 SET check_function_bodies = false;
7 SET client_min_messages = warning;
8 SET row_security = off;
9 CREATE EXTENSION IF NOT EXISTS plpgsql WITH SCHEMA pg_catalog;
10 COMMENT ON EXTENSION plpgsql IS 'PL/pgSQL procedural language';
11 SET search_path = public, pg_catalog;
12 SET default_tablespace = '';
13 SET default_with_oids = false;
14 CREATE TABLE speedtest_users (
15     id integer NOT NULL,
16     "timestamp" timestamp without time zone DEFAULT now() NOT NULL,
17     ip text NOT NULL,
18     ispinfo text,
19     extra text,
20     ua text NOT NULL,
21     lang text NOT NULL,
22     dl text,
23     ul text,
24     ping text,
25     jitter text,
```

```

26     log text
27 );
28 CREATE SEQUENCE speedtest_users_id_seq
29     START WITH 1
30     INCREMENT BY 1
31     NO MINVALUE
32     NO MAXVALUE
33     CACHE 1;
34 ALTER SEQUENCE speedtest_users_id_seq OWNED BY speedtest_users.id;
35 ALTER TABLE ONLY speedtest_users ALTER COLUMN id SET DEFAULT
    ↪ nextval('speedtest_users_id_seq'::regclass);
36 SELECT pg_catalog.setval('speedtest_users_id_seq', 1, true);
37 ALTER TABLE ONLY speedtest_users
38     ADD CONSTRAINT speedtest_users_pkey PRIMARY KEY (id);

```

Рисунок 10 — Скрипт инициализации базы данных

1.5 Сервис резервного копирования базы данных

Сервис `backup-db` выполняет периодическое создание дампов PostgreSQL и их загрузку в облачное S3-совместимое хранилище. Внутри контейнера работает cron с расписанием, задаваемым через переменную окружения. Dockerfile сервиса приведён на рисунке 11.

```

1 FROM debian:stable-slim
2
3 # Install required packages
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     wget unzip cron openssl ca-certificates postgresql-client \
6     && rm -rf /var/lib/apt/lists/*
7
8 # Install AWS CLI
9 RUN wget "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -O
    ↪ /tmp/awsliv2.zip \
10     && unzip /tmp/awsliv2.zip -d /tmp \
11     && /tmp/aws/install \
12     && rm -rf /tmp/aws /tmp/awsliv2.zip

```

```

13
14 # Download Globalsign Root R6 certificate
15 RUN mkdir -p /root/.certs \
16     && wget -q "https://secure.globalsign.net/cacert/root-r6.crt" -O
17     ↪ /root/.certs/root.crt \
18     && openssl x509 -inform der -in /root/.certs/root.crt -out
19     ↪ /root/.certs/root.crt \
20     && chmod 600 /root/.certs/root.crt
21
22 # Copy backup.sh
23 COPY backup.sh /usr/local/bin/backup.sh
24 RUN chmod +x /usr/local/bin/backup.sh
25
26 # ENV setup script
27 RUN echo "#!/bin/bash\nprintenv > /etc/environment" >
28     ↪ /usr/local/bin/setup-env.sh \
29     && chmod +x /usr/local/bin/setup-env.sh
30
31 # AWS & cron setup script
32 COPY docker-entrypoint.sh /
33 RUN chmod +x /docker-entrypoint.sh
34
35 ENTRYPOINT ["/docker-entrypoint.sh"]
36 CMD ["/bin/bash", "-c", "/usr/local/bin/setup-env.sh && cron -f"]

```

Рисунок 11 — Dockerfile для backup-db

Entrypoint-скрипт (рисунок 12) генерирует конфигурацию AWS CLI и cron-задание.

```

1 #!/bin/bash
2 set -e
3
4 mkdir -p /root/.aws
5
6 # Generating AWS CLI config
7 cat > /root/.aws/config << EOF
8 [default]
9 region = ${AWS_DEFAULT_REGION}

```

```

10 endpoint_url = ${AWS_ENDPOINT_URL}
11 ca_bundle = /root/.certs/root.crt
12 EOF
13
14 # Generating AWS credentials
15 cat > /root/.aws/credentials << EOF
16 [default]
17 aws_access_key_id = ${AWS_ACCESS_KEY_ID}
18 aws_secret_access_key = ${AWS_SECRET_ACCESS_KEY}
19 EOF
20
21 chmod 600 /root/.aws/credentials /root/.aws/config
22
23 # Setup cron timer
24 echo "${BACKUP_DB_CRON} root . /etc/environment; /usr/local/bin/backup.sh >>
  ↪ /proc/1/fd/1 2>&1" > /etc/cron.d/backup
25 chmod 0644 /etc/cron.d/backup
26 touch /var/log/backup.log
27
28 exec "$@"

```

Рисунок 12 — Entrypoint-скрипт backup-db

Сам скрипт создания дампа и загрузки в S3 показан на рисунке 13.

```

1  #!/bin/bash
2  set -e
3
4  POSTGRES_HOST="${POSTGRES_HOST:-postgres}"
5  POSTGRES_PORT="${POSTGRES_PORT:-5432}"
6  POSTGRES_USER="${POSTGRES_USER:-app_user}"
7  POSTGRES_DATABASE="${POSTGRES_DATABASE:-librespeed}"
8  S3_BUCKET_DB="${S3_BUCKET_DB}"
9
10 TIMESTAMP=$(date +"%Y-%m-%d-%H-%M")
11 BACKUP_FILE="/tmp/${TIMESTAMP}-${POSTGRES_DATABASE}.sql"
12 ARCHIVED_FILE="${BACKUP_FILE}.gz"
13
14 export PGPASSWORD="$POSTGRES_PASSWORD"

```

```

15
16 pg_dump -U "$POSTGRES_USER" -h "$POSTGRES_HOST" -p "$POSTGRES_PORT"
    ↪ "$POSTGRES_DATABASE" > "$BACKUP_FILE"
17 if [ $? -ne 0 ]; then
18     echo "Dump creation error"
19     exit 1
20 fi
21
22 gzip "$BACKUP_FILE"
23 if [ $? -ne 0 ]; then
24     echo "Dump compression error"
25     exit 1
26 fi
27
28 aws s3 cp "$ARCHIVED_FILE" "$S3_BUCKET_DB"
29 if [ $? -ne 0 ]; then
30     echo "Dump uploading to S3 error"
31     exit 1
32 fi
33
34 rm "$ARCHIVED_FILE"
35
36 echo "Backup completed: $(basename $ARCHIVED_FILE)"

```

Рисунок 13 — Скрипт резервного копирования БД

1.6 Сервис ротации и отправки логов nginx

Сервис `backup-logs` монтирует том с логами `nginx`, выполняет их ежедневную ротацию с помощью `logrotate` и по расписанию отправляет сжатые архивы в S3-хранилище. Dockerfile представлен на рисунке 14.

```

1 FROM debian:stable-slim
2
3 # Install required packages: awscli, cron, logrotate, wget, ca-certificates
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     wget unzip cron ca-certificates openssl logrotate docker.io \

```



```

6      && rm -rf /var/lib/apt/lists/*
7
8      # Install AWS CLI
9      RUN wget "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -O
        ↪ /tmp/awsclicv2.zip \
10      && unzip /tmp/awsclicv2.zip -d /tmp \
11      && /tmp/aws/install \
12      && rm -rf /tmp/aws /tmp/awsclicv2.zip
13
14      # Download Globalsign Root R6 certificate
15      RUN mkdir -p /root/.certs \
16      && wget -q "https://secure.globalsign.net/cacert/root-r6.crt" -O
        ↪ /root/.certs/root.crt \
17      && openssl x509 -inform der -in /root/.certs/root.crt -out
        ↪ /root/.certs/root.crt \
18      && chmod 600 /root/.certs/root.crt
19
20      # Copy bachup.sh
21      COPY backup-nginx-logs.sh /usr/local/bin/
22      RUN chmod +x /usr/local/bin/backup-nginx-logs.sh
23
24      # Copy logrotate.conf
25      COPY logrotate.conf /etc/logrotate.d/nginx
26      RUN chmod 644 /etc/logrotate.d/nginx
27
28      # ENV setup script
29      RUN echo "#!/bin/bash\nprintenv > /etc/environment" >
        ↪ /usr/local/bin/setup-env.sh \
30      && chmod +x /usr/local/bin/setup-env.sh
31
32      # AWS & cron setup script
33      COPY docker-entrypoint.sh /
34      RUN chmod +x /docker-entrypoint.sh
35
36      ENTRYPOINT ["/docker-entrypoint.sh"]
37      CMD ["/bin/bash", "-c", "/usr/local/bin/setup-env.sh && cron -f"]

```

Рисунок 14 — Dockerfile для backup-logs
 Конфигурация logrotate приведена на рисунке 15.

```
1 /var/log/nginx/*.log {
2     daily
3     missingok
4     rotate 14
5     compress
6     notifempty
7     copytruncate
8     sharedscripts
9 }
```

Рисунок 15 — Конфигурация logrotate
Entrypoint (рисунок 16) настраивает AWS CLI и cron.

```
1  #!/bin/bash
2  set -e
3
4  mkdir -p /root/.aws
5
6  # Generating AWS CLI config
7  cat > /root/.aws/config << EOF
8  [default]
9  region = ${AWS_DEFAULT_REGION}
10 endpoint_url = ${AWS_ENDPOINT_URL}
11 ca_bundle = /root/.certs/root.crt
12 EOF
13
14 # Generating AWS credentials
15 cat > /root/.aws/credentials << EOF
16 [default]
17 aws_access_key_id = ${AWS_ACCESS_KEY_ID}
18 aws_secret_access_key = ${AWS_SECRET_ACCESS_KEY}
19 EOF
20
21 chmod 600 /root/.aws/credentials /root/.aws/config
22
23 # Setup cron timer
24 echo "${BACKUP_LOGS_CRON} root . /etc/environment; \
```

```

25     /usr/local/bin/backup-nginx-logs.sh /var/log/nginx
    ↪ /var/log/backup-nginx-logs.log; \
26     docker kill --signal=HUP nginx >> /proc/1/fd/1 2>&1" > /etc/cron.d/backup
27 chmod 0644 /etc/cron.d/backup
28 touch /var/log/backup.log
29
30 exec "$@"

```

Рисунок 16 — Entrypoint-скрипт backup-logs
Скрипт отправки логов в S3 показан на рисунке 17.

```

1  #!/bin/bash
2
3  LOG_DIR="$BACKUP_LOGS_VOLUME"
4  S3_BUCKET_LOGS="${S3_BUCKET_LOGS}"
5
6  log() {
7      echo "$(date '+%d.%m.%Y %H:%M:%S') [$1] $2"
8  }
9
10 upload() {
11     local loc_archive="$1"
12     local s3_dir_time=$(date -r "$loc_archive" '+%Y/%m/%d')
13     local s3_path="${S3_BUCKET_LOGS}${s3_dir_time}/"
14
15     local loc_hash="${loc_archive}.sha256"
16     (cd "$(dirname "$loc_archive")" && sha256sum "$loc_archive") > "$loc_hash"
17
18     if ! aws s3 cp "$loc_archive" "$s3_path"; then
19         log "ERROR" "file upload failed: $loc_archive"
20         rm -f "$loc_hash"
21         return 1
22     fi
23
24     if ! aws s3 cp "$loc_hash" "$s3_path"; then
25         log "ERROR" "file upload failed: $loc_hash"
26         rm -f "$loc_hash"
27         return 1

```

```
28     fi
29
30     rm -f "$loc_archive" "$loc_hash"
31     log "INFO" "file and hash were uploaded: $loc_archive"
32     return 0
33 }
34
35 log "INFO" "uploader started in $LOG_DIR"
36 find "$LOG_DIR" -maxdepth 1 -type f -name "*.gz" -print0 | while IFS= read -r
37 ↪ -d '' gzfile; do
38     upload "$gzfile"
39 done
```

Рисунок 17 — Скрипт отправки логов nginx в S3

2 ТЕСТИРОВАНИЕ

2.1 Проверка работоспособности сайта

На рисунке 18 показан работающий сайт LibreSpeed.

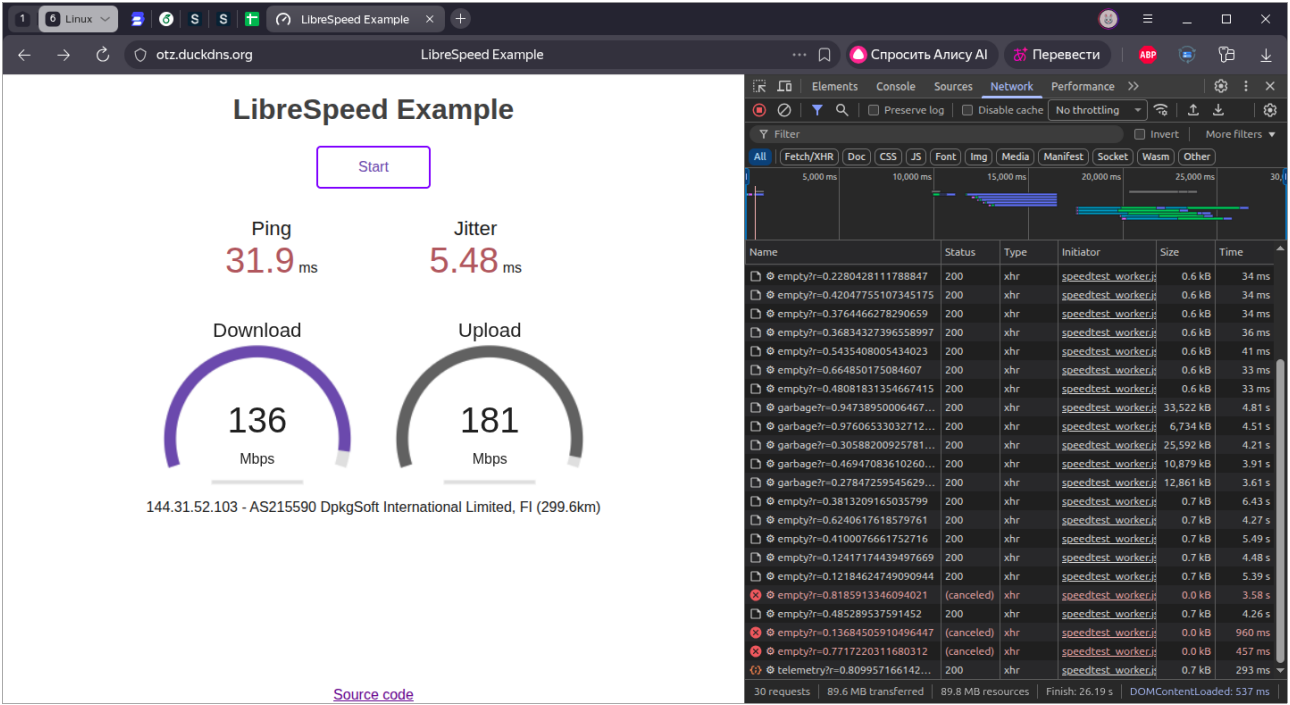


Рисунок 18 — Главная страница LibreSpeed

Страница ошибки, отдаваемая nginx, показана на рисунке 19.

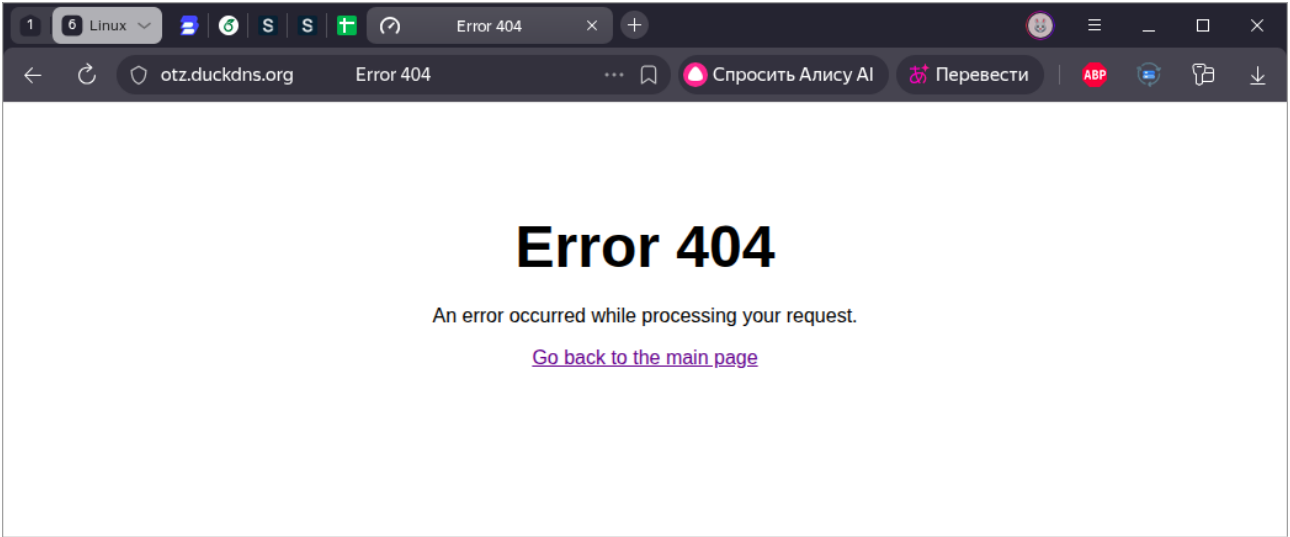
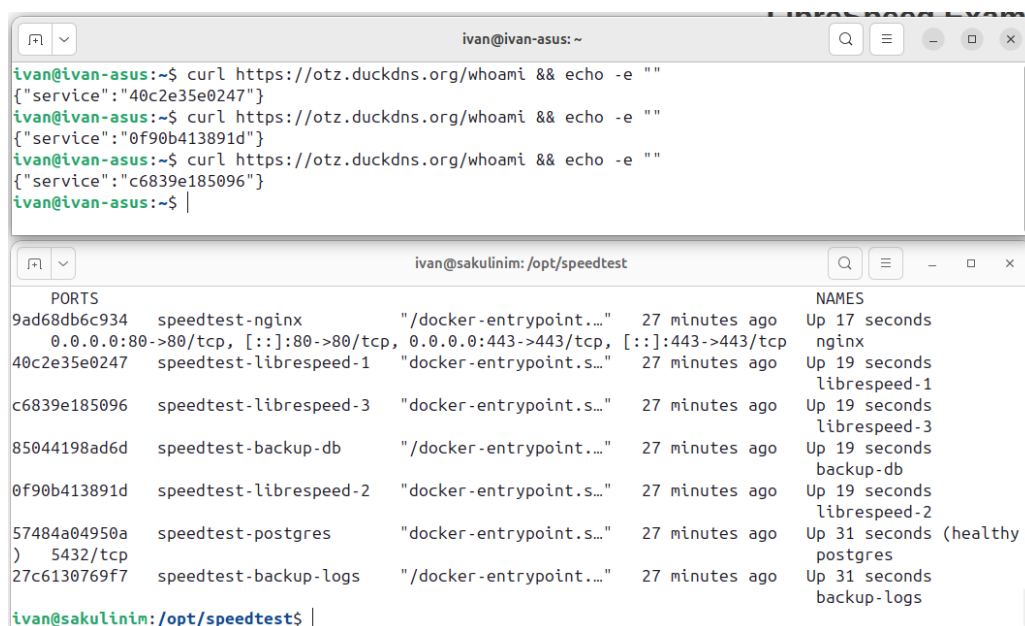


Рисунок 19 — Кастомная страница ошибки 404

2.2 Отказоустойчивость при падении сервисов

Для проверки балансировки и отказоустойчивости был остановлен один из контейнеров `librespeed-1`. На рисунках 20–22 видно, что сайт продолжает функционировать, а запросы распределяются между оставшимися двумя экземплярами.

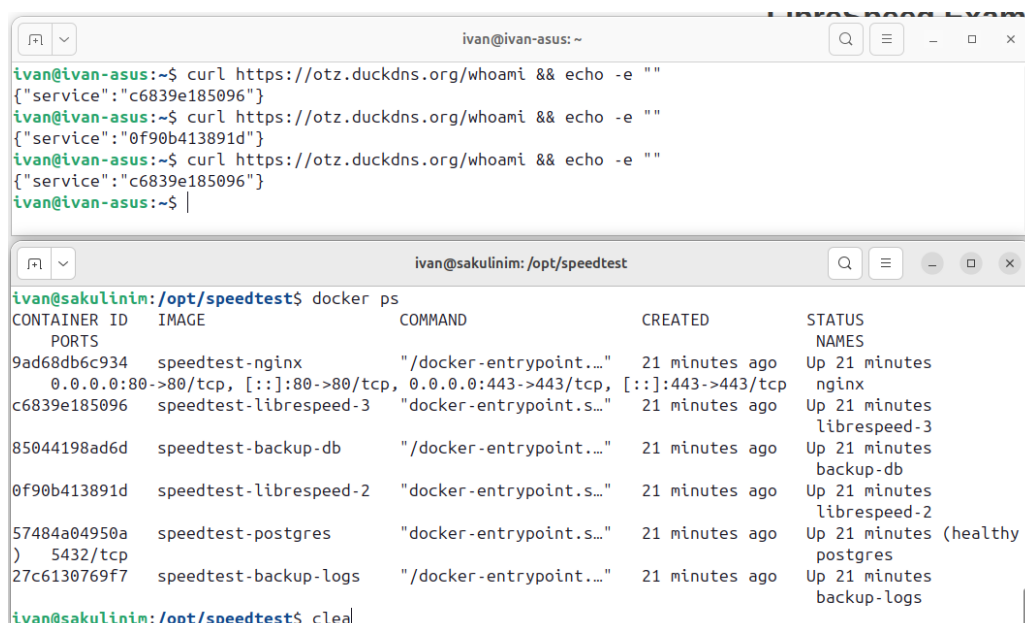


The image shows two terminal windows. The top window, titled 'ivan@ivan-asus: ~', shows three successful curl commands to 'https://otz.duckdns.org/whoami' returning JSON responses with service IDs. The bottom window, titled 'ivan@sakulinim: /opt/speedtest', shows the output of a 'docker ps' command. It lists several containers, including 'speedtest-librespeed-1' which is shown as 'Up 19 seconds'.

```
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"40c2e35e0247"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"0f90b413891d"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ |

ivan@sakulinim: /opt/speedtest$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS        NAMES
9ad68db6c934   speedtest-nginx    "/docker-entrypoint..." 27 minutes ago Up 17 seconds nginx
0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
40c2e35e0247   speedtest-librespeed-1 "docker-entrypoint.s..." 27 minutes ago Up 19 seconds librespeed-1
c6839e185096   speedtest-librespeed-3 "docker-entrypoint.s..." 27 minutes ago Up 19 seconds librespeed-3
85044198ad6d   speedtest-backup-db "/docker-entrypoint..." 27 minutes ago Up 19 seconds backup-db
0f90b413891d   speedtest-librespeed-2 "docker-entrypoint.s..." 27 minutes ago Up 19 seconds librespeed-2
57484a04950a   speedtest-postgres "docker-entrypoint.s..." 27 minutes ago Up 31 seconds (healthy) postgres
) 5432/tcp
27c6130769f7   speedtest-backup-logs "/docker-entrypoint..." 27 minutes ago Up 31 seconds backup-logs
ivan@sakulinim: /opt/speedtest$ |
```

Рисунок 20 — Распределение при трёх контейнерах



The image shows two terminal windows. The top window, titled 'ivan@ivan-asus: ~', shows three successful curl commands to 'https://otz.duckdns.org/whoami' returning JSON responses with service IDs. The bottom window, titled 'ivan@sakulinim: /opt/speedtest', shows the output of a 'docker ps' command. It lists several containers, including 'speedtest-librespeed-3' which is shown as 'Up 21 minutes'.

```
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"0f90b413891d"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ |

ivan@sakulinim: /opt/speedtest$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS        NAMES
9ad68db6c934   speedtest-nginx    "/docker-entrypoint..." 21 minutes ago Up 21 minutes nginx
0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
c6839e185096   speedtest-librespeed-3 "docker-entrypoint.s..." 21 minutes ago Up 21 minutes librespeed-3
85044198ad6d   speedtest-backup-db "/docker-entrypoint..." 21 minutes ago Up 21 minutes backup-db
0f90b413891d   speedtest-librespeed-2 "docker-entrypoint.s..." 21 minutes ago Up 21 minutes librespeed-2
57484a04950a   speedtest-postgres "docker-entrypoint.s..." 21 minutes ago Up 21 minutes (healthy) postgres
) 5432/tcp
27c6130769f7   speedtest-backup-logs "/docker-entrypoint..." 21 minutes ago Up 21 minutes backup-logs
ivan@sakulinim: /opt/speedtest$ clea|
```

Рисунок 21 — Остановка одного контейнера `librespeed`

```
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ curl https://otz.duckdns.org/whoami && echo -e ""
{"service":"c6839e185096"}
ivan@ivan-asus:~$ |

ivan@sakulinim:/opt/speedtest$ docker kill 0f90b413891d
0f90b413891d
ivan@sakulinim:/opt/speedtest$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        NAMES
9ad68db6c934   speedtest-nginx      "/docker-entrypoint..." 25 minutes ago Up 25 minutes nginx
0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
c6839e185096   speedtest-librespeed-3 "docker-entrypoint.s..." 25 minutes ago Up 25 minutes librespeed-3
85044198ad6d   speedtest-backup-db  "/docker-entrypoint..." 25 minutes ago Up 25 minutes backup-db
57484a04950a   speedtest-postgres   "docker-entrypoint.s..." 25 minutes ago Up 25 minutes (healthy) postgres
27c6130769f7   speedtest-backup-logs "/docker-entrypoint..." 25 minutes ago Up 25 minutes backup-logs
ivan@sakulinim:/opt/speedtest$
```

Рисунок 22 — Остановка двух контейнеров librespeed

2.3 Мониторинг состояния контейнеров

Вывод команд `docker compose ps` и `docker compose stats` демонстрирует статус всех сервисов и потребляемые ресурсы (рисунки 23 и 24).

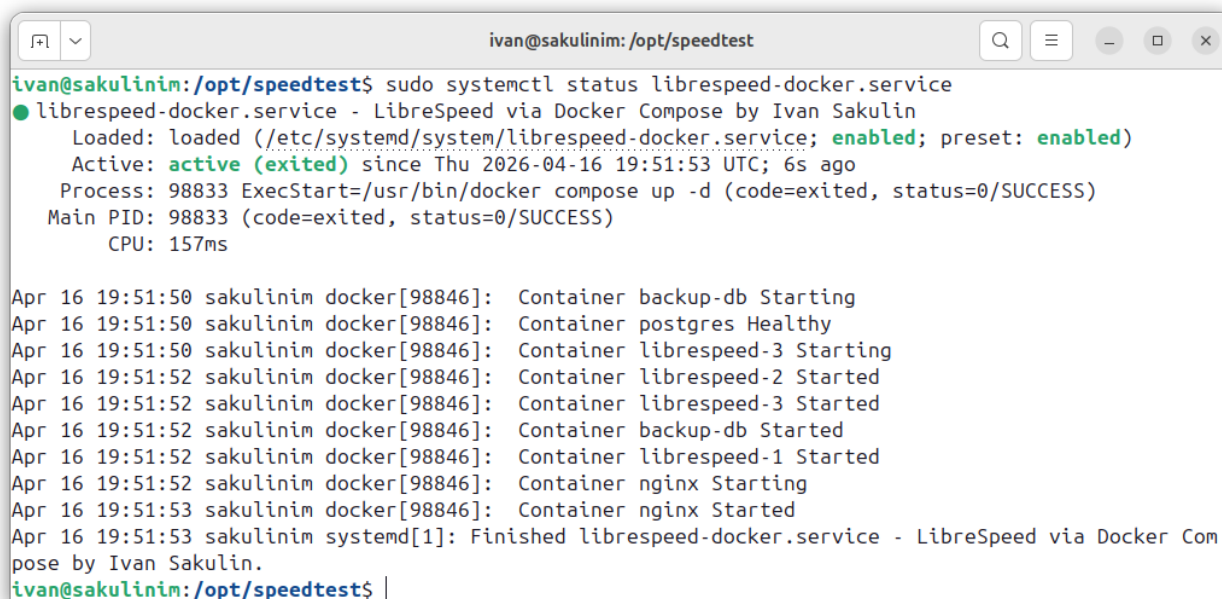
```
ivan@sakulinim:/opt/speedtest$ docker compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED        STATUS        PORTS
backup-db           speedtest-backup-db  "/docker-entrypoint..." backup-db   About a minute ago Up About a minute
backup-logs         speedtest-backup-logs "/docker-entrypoint..." backup-logs About a minute ago Up About a minute
librespeed-1        speedtest-librespeed-1 "docker-entrypoint.s..." librespeed-1 About a minute ago Up About a minute
librespeed-2        speedtest-librespeed-2 "docker-entrypoint.s..." librespeed-2 About a minute ago Up About a minute
librespeed-3        speedtest-librespeed-3 "docker-entrypoint.s..." librespeed-3 About a minute ago Up About a minute
nginx               speedtest-nginx      "/docker-entrypoint..." nginx      About a minute ago Up About a minute
0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
postgres            speedtest-postgres   "docker-entrypoint.s..." postgres   About a minute ago Up About a minute (healthy) 5432/tcp
ivan@sakulinim:/opt/speedtest$
```

Рисунок 23 — Статус контейнеров (`docker compose ps`)

```
ivan@sakulinim:/opt/speedtest$ docker compose stats
CONTAINER ID   NAME                CPU %    MEM USAGE / LIMIT   MEM %    NET I/O    BLOCK I/O  PIDS
9ad68db6c934   nginx               0.00%    3.594MiB / 512MiB    0.70%    1.01kB / 126B    24.6kB / 4.1kB    3
40c2e35e0247   librespeed-1        0.00%    24.73MiB / 3.824GiB  0.63%    1.17kB / 126B    0B / 0B          7
c6839e185096   librespeed-3        0.00%    24.71MiB / 3.824GiB  0.63%    1.26kB / 126B    0B / 0B          7
85044198ad6d   backup-db           0.01%    604KiB / 3.824GiB    0.02%    1.22kB / 126B    0B / 20.5kB       1
0f90b413891d   librespeed-2        0.00%    24.75MiB / 3.824GiB  0.63%    1.3kB / 126B     0B / 0B          7
57484a04950a   postgres            0.01%    18.75MiB / 3.824GiB  0.48%    1.93kB / 126B    0B / 606kB        6
27c6130769f7   backup-logs         0.01%    620KiB / 3.824GiB    0.02%    2.08kB / 126B    0B / 20.5kB       1
```

Рисунок 24 — Потребление ресурсов (`docker compose stats`)

Systemd-сервис `librespeed-docker` находится в активном состоянии и включён для автозапуска (рисунок 25).

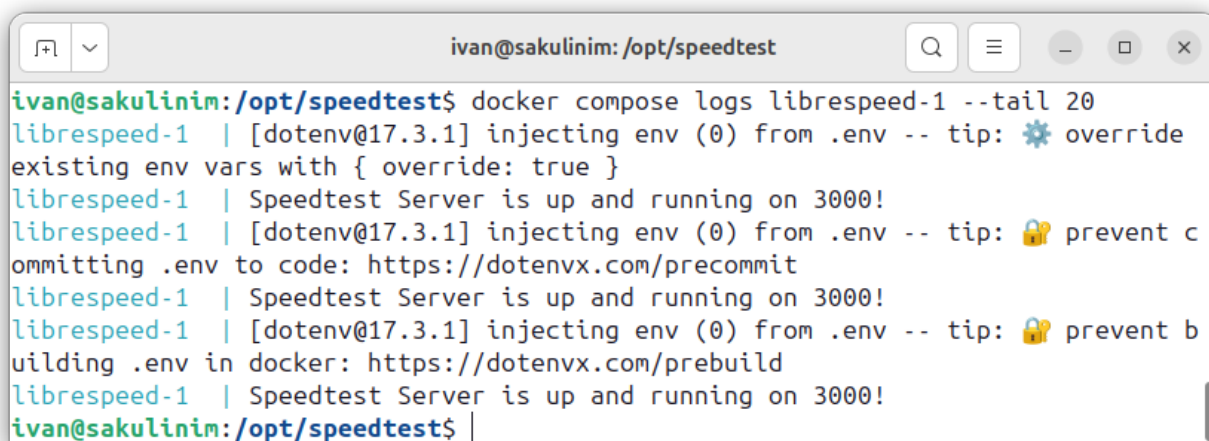


```
ivan@sakulinim: /opt/speedtest
ivan@sakulinim:/opt/speedtest$ sudo systemctl status librespeed-docker.service
● librespeed-docker.service - LibreSpeed via Docker Compose by Ivan Sakulin
   Loaded: loaded (/etc/systemd/system/librespeed-docker.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2026-04-16 19:51:53 UTC; 6s ago
     Process: 98833 ExecStart=/usr/bin/docker compose up -d (code=exited, status=0/SUCCESS)
    Main PID: 98833 (code=exited, status=0/SUCCESS)
       CPU: 157ms

Apr 16 19:51:50 sakulinim docker[98846]: Container backup-db Starting
Apr 16 19:51:50 sakulinim docker[98846]: Container postgres Healthy
Apr 16 19:51:50 sakulinim docker[98846]: Container librespeed-3 Starting
Apr 16 19:51:52 sakulinim docker[98846]: Container librespeed-2 Started
Apr 16 19:51:52 sakulinim docker[98846]: Container librespeed-3 Started
Apr 16 19:51:52 sakulinim docker[98846]: Container backup-db Started
Apr 16 19:51:52 sakulinim docker[98846]: Container librespeed-1 Started
Apr 16 19:51:52 sakulinim docker[98846]: Container nginx Starting
Apr 16 19:51:53 sakulinim docker[98846]: Container nginx Started
Apr 16 19:51:53 sakulinim systemd[1]: Finished librespeed-docker.service - LibreSpeed via Docker Compose by Ivan Sakulin.
ivan@sakulinim:/opt/speedtest$
```

Рисунок 25 — Статус systemd-сервиса

Сервис работает без ошибок, что подтверждают логи (рисунок 26).



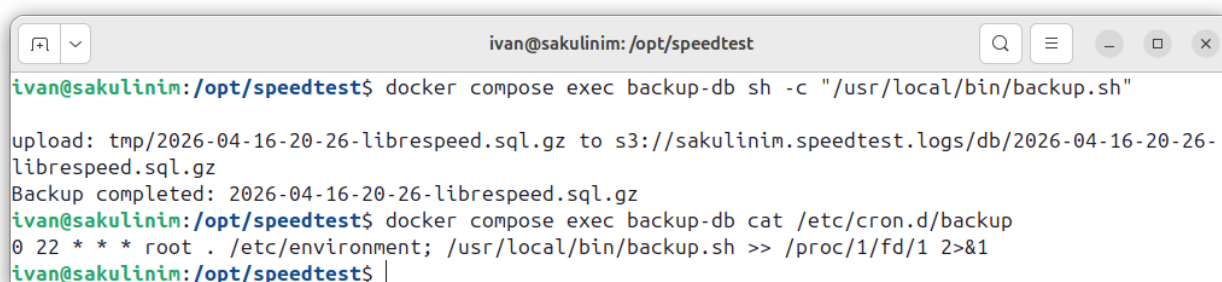
```
ivan@sakulinim: /opt/speedtest
ivan@sakulinim:/opt/speedtest$ docker compose logs librespeed-1 --tail 20
librespeed-1 | [dotenv@17.3.1] injecting env (0) from .env -- tip: ⚙️ override
librespeed-1 | Speedtest Server is up and running on 3000!
librespeed-1 | [dotenv@17.3.1] injecting env (0) from .env -- tip: 🗝️ prevent c
ommitting .env to code: https://dotenvx.com/precommit
librespeed-1 | Speedtest Server is up and running on 3000!
librespeed-1 | [dotenv@17.3.1] injecting env (0) from .env -- tip: 🗝️ prevent b
uilding .env in docker: https://dotenvx.com/prebuild
librespeed-1 | Speedtest Server is up and running on 3000!
ivan@sakulinim:/opt/speedtest$
```

Рисунок 26 — Журнал контейнера librespeed-1

2.4 Тестирование резервного копирования

2.4.1 Резервное копирование базы данных

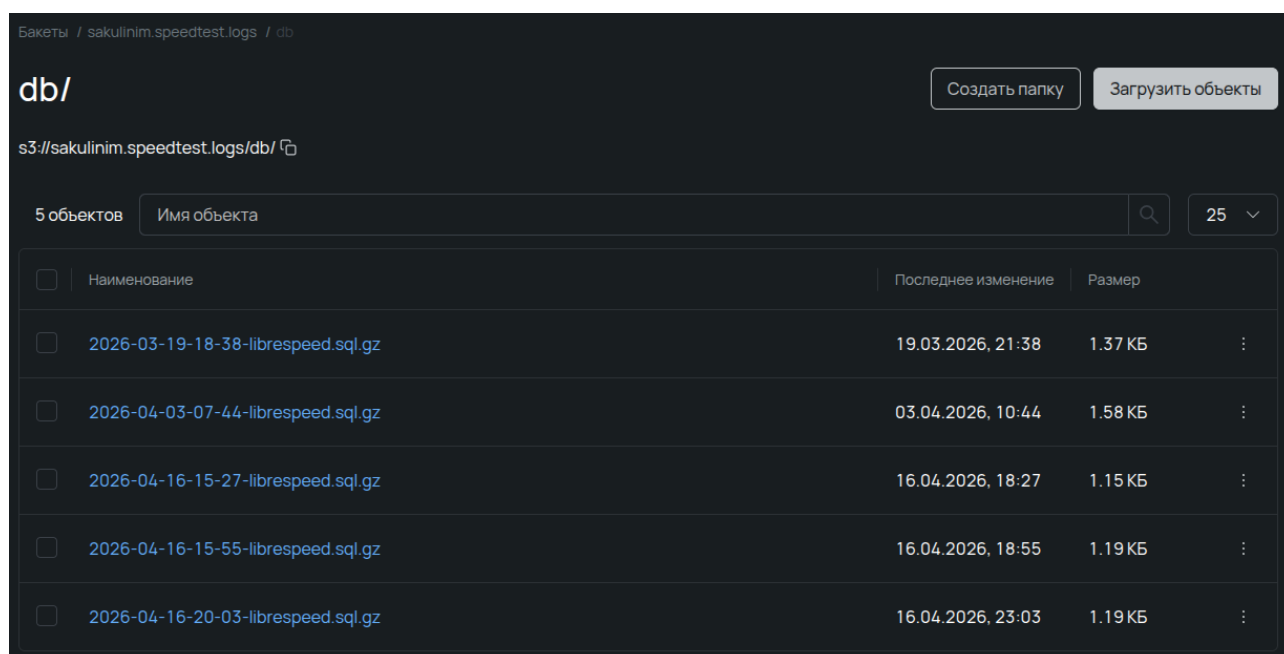
Ручной запуск скрипта `backup.sh` внутри контейнера `backup-db` подтверждает создание дампа и его успешную загрузку в S3-хранилище (рисунок 27).



```
ivan@sakulinim: /opt/speedtest
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-db sh -c "/usr/local/bin/backup.sh"
upload: tmp/2026-04-16-20-26-librespeed.sql.gz to s3://sakulinim.speedtest.logs/db/2026-04-16-20-26-librespeed.sql.gz
Backup completed: 2026-04-16-20-26-librespeed.sql.gz
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-db cat /etc/cron.d/backup
0 22 * * * root . /etc/environment; /usr/local/bin/backup.sh >> /proc/1/fd/1 2>&1
ivan@sakulinim:/opt/speedtest$
```

Рисунок 27 — Ручной запуск `backup.sh`

На рисунке 28 показан загруженный файл дампа в веб-интерфейсе облачного хранилища Selectel.



Бакеты / sakulinim.speedtest.logs / db

db/ Создать папку Загрузить объекты

s3://sakulinim.speedtest.logs/db/

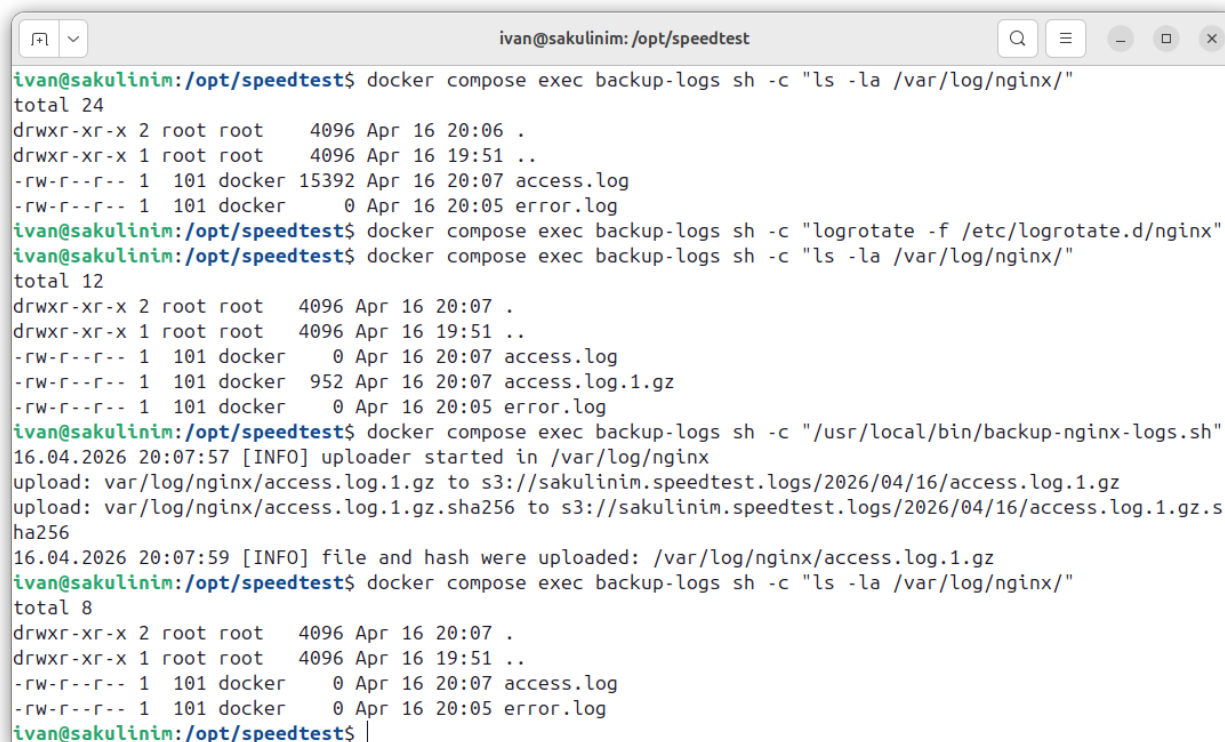
5 объектов 25

☐	Наименование	Последнее изменение	Размер	
☐	2026-03-19-18-38-librespeed.sql.gz	19.03.2026, 21:38	1.37 КБ	
☐	2026-04-03-07-44-librespeed.sql.gz	03.04.2026, 10:44	1.58 КБ	
☐	2026-04-16-15-27-librespeed.sql.gz	16.04.2026, 18:27	1.15 КБ	
☐	2026-04-16-15-55-librespeed.sql.gz	16.04.2026, 18:55	1.19 КБ	
☐	2026-04-16-20-03-librespeed.sql.gz	16.04.2026, 23:03	1.19 КБ	

Рисунок 28 — Файл дампа в S3-бакете

2.4.2 Ротация и отправка логов nginx

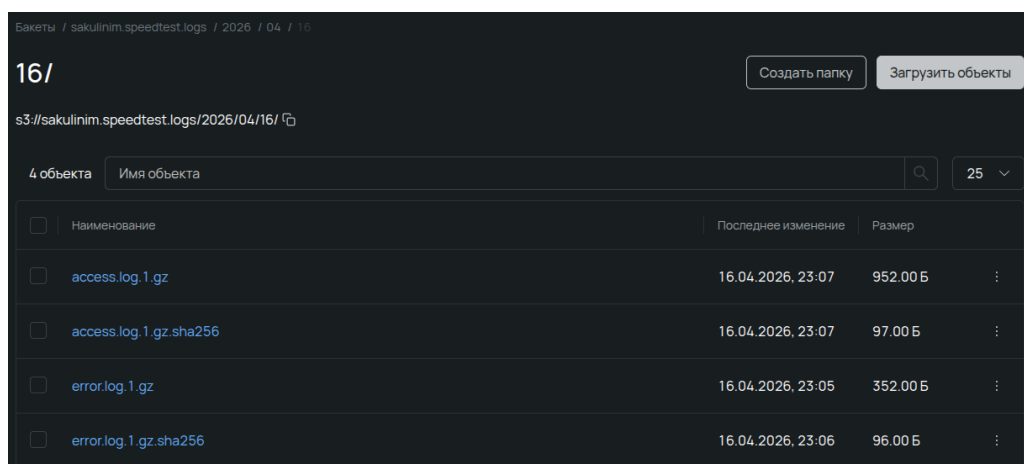
Ручной запуск скрипта `backup-nginx-logs.sh` (рисунок 29) демонстрирует обработку сжатых логов и их загрузку в S3.



```
ivan@sakulinim: /opt/speedtest
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-logs sh -c "ls -la /var/log/nginx/"
total 24
drwxr-xr-x 2 root root 4096 Apr 16 20:06 .
drwxr-xr-x 1 root root 4096 Apr 16 19:51 ..
-rw-r--r-- 1 101 docker 15392 Apr 16 20:07 access.log
-rw-r--r-- 1 101 docker 0 Apr 16 20:05 error.log
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-logs sh -c "logrotate -f /etc/logrotate.d/nginx"
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-logs sh -c "ls -la /var/log/nginx/"
total 12
drwxr-xr-x 2 root root 4096 Apr 16 20:07 .
drwxr-xr-x 1 root root 4096 Apr 16 19:51 ..
-rw-r--r-- 1 101 docker 0 Apr 16 20:07 access.log
-rw-r--r-- 1 101 docker 952 Apr 16 20:07 access.log.1.gz
-rw-r--r-- 1 101 docker 0 Apr 16 20:05 error.log
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-logs sh -c "/usr/local/bin/backup-nginx-logs.sh"
16.04.2026 20:07:57 [INFO] uploader started in /var/log/nginx
upload: var/log/nginx/access.log.1.gz to s3://sakulinim.speedtest.logs/2026/04/16/access.log.1.gz
upload: var/log/nginx/access.log.1.gz.sha256 to s3://sakulinim.speedtest.logs/2026/04/16/access.log.1.gz.s
ha256
16.04.2026 20:07:59 [INFO] file and hash were uploaded: /var/log/nginx/access.log.1.gz
ivan@sakulinim:/opt/speedtest$ docker compose exec backup-logs sh -c "ls -la /var/log/nginx/"
total 8
drwxr-xr-x 2 root root 4096 Apr 16 20:07 .
drwxr-xr-x 1 root root 4096 Apr 16 19:51 ..
-rw-r--r-- 1 101 docker 0 Apr 16 20:07 access.log
-rw-r--r-- 1 101 docker 0 Apr 16 20:05 error.log
ivan@sakulinim:/opt/speedtest$
```

Рисунок 29 — Ручной запуск скрипта отправки логов

На рисунке 30 видно, что файлы логов, сгруппированные по датам, успешно размещены в S3-бакете.



Бакеты / sakulinim.speedtest.logs / 2026 / 04 / 16			
16/		Создать папку	Загрузить объекты
s3://sakulinim.speedtest.logs/2026/04/16/			
4 объекта			
Имя объекта			
☐	Наименование	Последнее изменение	Размер
☐	access.log.1.gz	16.04.2026, 23:07	952.00 Б
☐	access.log.1.gz.sha256	16.04.2026, 23:07	97.00 Б
☐	error.log.1.gz	16.04.2026, 23:05	352.00 Б
☐	error.log.1.gz.sha256	16.04.2026, 23:06	96.00 Б

Рисунок 30 — Архивы логов nginx в S3-хранилище

3 ВОПРОСЫ

1) Почему при переходе на Docker Compose мы убрали директиву `root` из конфигурации Nginx?

В исходной конфигурации статические ресурсы размещались на том же сервере, и директива `root` указывала путь к файлам. При переходе на контейнеризацию основное приложение LibreSpeed проксируется на бэкенды, которые самостоятельно отдают статику через `express.static`, а для отдельных файлов (`favicon.ico`, `error.html`) используется отдельный `location` с собственным `root` внутри образа Nginx. Таким образом, глобальная директива `root` стала избыточной и была удалена для упрощения конфигурации.

2) Каким образом происходит адресация Docker Compose?

В пользовательской сети, создаваемой Docker Compose, каждый сервис получает DNS-имя, совпадающее с именем сервиса в файле `docker-compose.yml`. Встроенный DNS-сервер автоматически разрешает эти имена в IP-адреса соответствующих контейнеров, позволяя сервисам обращаться друг к другу по понятным псевдонимам (например, `postgres`, `librespeed-1`) без необходимости жёстко задавать IP-адреса.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы инфраструктура, ранее развёрнутая на системных процессах Linux, была полностью переведена в контейнеризованное окружение с использованием Docker Compose. Разработаны собственные Dockerfile для каждого компонента: веб-сервера nginx, трёх экземпляров приложения LibreSpeed, базы данных PostgreSQL и сервисов резервного копирования. Настроены тома для хранения данных БД и логов, реализованы проверки работоспособности (healthcheck) и зависимости между сервисами. Обеспечено автоматическое возобновление работы стека при перезагрузке сервера через systemd-юнит. Проведённое тестирование подтвердило корректность балансировки запросов, отказоустойчивость, создание резервных копий базы данных и логов по расписанию с последующей отправкой в облачное S3-хранилище.